

Field Project Report
on
OptiPick – A Smart Purchase Advisor

Course/Program:

SY BSc.IT

Date:

8th September, 2025

Academic Year:

2025-26

Table of Contents

CONTENT	PAGE NO	DATE	SIGNATURE
1. INTRODUCTION		01/09/2025	
1.1 Project Overview	4		
1.2 Objectives	4		
1.3 Scope	5		
2. LITERATURE REVIEW		01/09/2025	
2.1 Background Information	7		
2.2 Related Work	9		
3. METHODOLOGY		02/09/2025	
3.1 Development Approach	12		
3.2 Tools & Technologies Used	13		
4. SYSTEM DESIGN		02/09/2025	
4.1 Architecture	15		
4.2 Diagrams (UML, Flowcharts)	16		
5. IMPLEMENTATION		02/09/2025	
5.1 Code Structure	18		
5.2 Key Features	28		
6. DEPLOYMENT		03/09/2025	
6.1 Deployment Process	29		
6.2 User Documentation	29		
7. RESULTS AND DISCUSSION		04/09/2025	
7.1 Achievements	32		

7.2 Challenges Faced	34		
8. CONCLUSION		04/09/2025	
8.1 Summary of Findings	35		
8.2 Future Work	36		
9. REFERENCES	38	05/09/2025	
10. APPENDICES		06/09/2025	
Appendix A: GUI Screenshots	39		
Appendix B: Sample Input & Output Data (Json File)	40		
Appendix C: Feature Weights Table	42		

1. INTRODUCTION

1.1 Project Overview:

In today's world, individuals are constantly faced with the challenge of making decisions from a wide range of alternatives. Whether purchasing a smartphone, selecting a course, or choosing a career path, the abundance of available options often results in decision fatigue and confusion. With the rise of e-commerce platforms, product comparisons have become even more complex, as customers must evaluate not only prices but also specifications, features, and long-term value.

Recommendation systems powered by Artificial Intelligence (AI) have emerged as a solution to this problem. Large-scale platforms like Amazon, Flipkart, Netflix, and Spotify use AI to process massive datasets, understand user behaviour, and deliver highly personalized suggestions. However, these solutions require continuous internet connectivity, large computational resources, and access to vast datasets. Not all users or institutions, especially in offline or resource-constrained environments can benefit from such sophisticated systems.

OptiPick – A Smart Purchase Advisor was conceptualized to fill this gap. Unlike large-scale AI systems, *OptiPick* (which stands for Optimal Pick) provides a lightweight, offline decision-support application that can operate without internet dependency. It allows users to manually input product details, such as budget, price, and key features, and applies a scoring algorithm to generate data-driven recommendations. The purpose is to help users make informed, budget-friendly, and logical choices while minimizing manual effort and guesswork.

The significance of *OptiPick* lies in its dual role:

1. As a practical tool, it supports users in everyday decision-making by providing clear and unbiased comparisons.
2. As a foundation for future AI integration, it opens possibilities for incorporating machine learning, predictive modelling, and natural language processing to evolve into a fully AI-enabled recommender system.

Thus, *OptiPick* bridges the gap between manual comparisons and advanced AI systems, offering a structured, user-friendly, and offline solution that empowers individuals to make smarter choices.

1.2 Objectives:

The primary objective of *OptiPick* is to design and develop an offline recommendation system that provides users with accurate, structured, and unbiased product comparisons. This central goal is supported by the following specific objectives:

1. To simplify product selection and decision-making:
 - Reduce the confusion caused by multiple alternatives by presenting results in a clear, structured manner.
 - Offer logical recommendations that consider both price and features.
2. To integrate budget management with recommendations:
 - Allow users to define a spending limit.
 - Ensure the best possible product is suggested within the available budget.
3. To implement a scoring algorithm for fairness and accuracy:
 - Rank products by assigning weighted scores based on user-defined priorities.
 - Reduce bias by ensuring every feature is considered systematically.
4. To provide visual representation of data:
 - Use charts, graphs, and tables for better understanding of comparisons.
 - Make decision-making more interactive and less time-consuming.
5. To support data storage and continuity:
 - Enable users to load and save their product lists in JSON format.
 - Allow reports to be generated in CSV format for future reference or sharing.
6. To create a base for future AI integration:
 - Lay groundwork for machine learning features such as predictive analysis and automated preference detection.
 - Prepare the system for possible integration with cloud and real-time APIs.

By fulfilling these objectives, *OptiPick* ensures it is not only functional in its current form but also adaptable to future AI-driven advancements.

1.3 Scope:

The scope of *OptiPick* reflects both its current deliverables and its strategic direction toward AI integration. In its present stage, the system operates as a rule-based recommender, where recommendations are generated using a transparent scoring algorithm based on user input, budget considerations, and feature weights. This ensures that the process is logical, explainable, and reliable for end-users, even in offline settings.

At the same time, the project is intentionally designed with a forward-looking perspective. While rule-based systems provide clarity and control, they lack the adaptive intelligence of modern AI recommenders. By positioning itself as a scalable prototype, *OptiPick* establishes the groundwork for incorporating AI techniques such as machine learning, natural language processing, and predictive analytics in the future.

Currently, *OptiPick* offers:

- Offline accessibility, ensuring usability without internet dependency.
- Structured product management, where users can input, prioritize, and evaluate multiple alternatives.
- Budget-sensitive recommendations, balancing cost with feature quality.
- Algorithm-driven scoring and ranking, minimizing bias and subjectivity.
- Data storage and export, with JSON and CSV support for continuity.
- Visualization tools, including tables and charts for transparent interpretation of results.

In the long term, *OptiPick* envisions a transition from a rule-based approach to an AI-empowered ecosystem, with enhancements such as:

- Adaptive learning models, where the system improves recommendations based on past user behaviour.
- Natural language interaction, allowing users to input queries conversationally.
- Predictive insights, forecasting product durability, popularity, or long-term value using data trends.
- Cross-platform deployment, extending usability to mobile and web applications.

By defining its scope in this way, *OptiPick* is presented not as a limited project, but as a robust current solution with a clear roadmap toward AI-driven innovation. It demonstrates academic value today while positioning itself for scalable growth in the evolving landscape of intelligent recommender systems.

2. LITERATURE REVIEW

2.1 Background Information:

In the current digital era, consumers face the constant challenge of information overload. With e-commerce platforms offering thousands of alternatives across similar product categories, selecting the most suitable option has become increasingly difficult. This problem is not limited to online shopping; even in offline contexts, individuals often struggle to compare alternatives objectively within the constraints of budget, usability, and personal preference.

To address such challenges, the concept of Decision Support Systems (DSS) emerged. A DSS provides structured tools and methodologies to assist individuals in making informed decisions by evaluating different parameters, applying scoring models, or simulating scenarios. Within this larger field, Recommendation Systems have developed as a specialized branch, focusing on guiding users toward the most relevant options among a large set of alternatives.

Recommendation systems can broadly be divided into two categories:

1. Rule-based Recommendation Systems:
These operate using predefined rules, scoring models, or filtering mechanisms. They are transparent and easy to explain because users can clearly understand why a recommendation was made. However, their rigidity often prevents adaptability, as they cannot learn from user behaviour or adjust dynamically to changing contexts.
2. AI-driven Recommendation Systems:
These rely on techniques such as machine learning, collaborative filtering, content-based filtering, and hybrid approaches. Unlike rule-based systems, they adapt to user preferences and behaviours over time, producing personalized recommendations. AI-powered recommenders dominate in industries such as e-commerce (Amazon, Flipkart), entertainment (Netflix, Spotify, YouTube), and education (Coursera, Udemy).

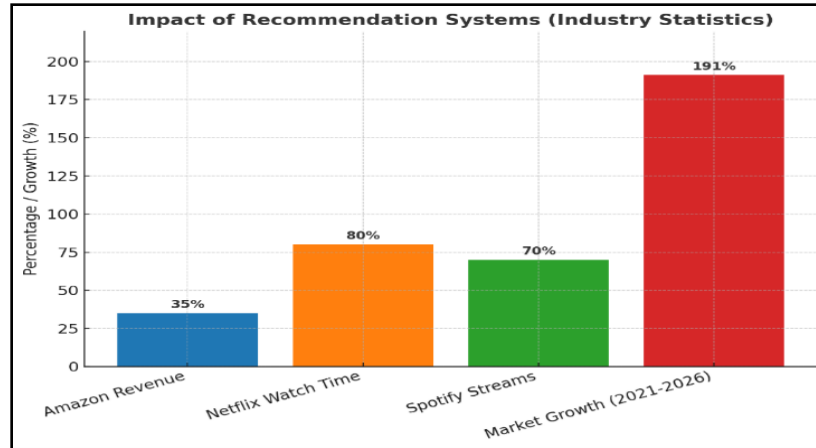


Figure 2.1.1: Industry reliance on AI-driven recommendation systems (Amazon, Netflix, Spotify, Market Growth 2021–2026).

As shown in the chart, major platforms derive a significant share of their engagement and revenue from recommendation systems- Amazon generates 35% of its revenue from recommendations, Netflix attributes 80% of viewing activity to its algorithms, and Spotify relies on AI playlists for nearly 70% of user engagement. On a global scale, the recommendation systems market is projected to grow by more than 190% between 2021 and 2026, highlighting their central role in consumer decision-making.

Despite their success, AI-driven recommenders face several challenges:

- **Opacity (Black Box Problem):** Many machine learning models lack explainability, making it difficult for users to understand why a particular suggestion was generated.
- **Data Dependency:** These systems require massive datasets to be effective, limiting their applicability in small-scale or offline contexts.
- **Resource Requirements:** High computational power and constant internet connectivity are often necessary.

The current body of knowledge thus presents a dual reality: rule-based systems are transparent and lightweight but limited in adaptability, while AI-driven systems are powerful but resource-heavy and opaque. This duality opens a space for solutions like *OptiPick*, which seek to combine the clarity of rule-based methods with the scalability of AI frameworks in the future.

2.2 Related Work:

Research and practice in recommendation systems have evolved significantly over the past two decades, spanning large-scale industrial deployments and academic prototypes. Together, they provide valuable insights into the capabilities and limitations of different approaches, while also revealing gaps that solutions like *OptiPick* aim to address.

1. Industrial Implementations:

In industry, recommendation systems are a cornerstone of user engagement and revenue generation. **E-commerce platforms** such as Amazon, Flipkart, and eBay employ AI-driven collaborative filtering and hybrid models to recommend products based on browsing history, purchase behaviour, and product similarities. These systems have demonstrated remarkable success; for instance, Amazon attributes over one-third of its sales to its recommender engine. However, the reliance on massive datasets and high computational resources makes them less suitable for smaller-scale or offline applications.

In the **entertainment sector**, companies like Netflix, Spotify, and YouTube use advanced machine learning models to deliver personalized experiences. Netflix applies collaborative filtering and deep learning to generate content suggestions, while Spotify combines content-based filtering with social and behavioral data to curate playlists such as “Discover Weekly.” These implementations highlight the effectiveness of AI-driven recommenders in improving engagement, but they also introduce concerns such as algorithmic opacity, privacy risks, and scalability challenges.

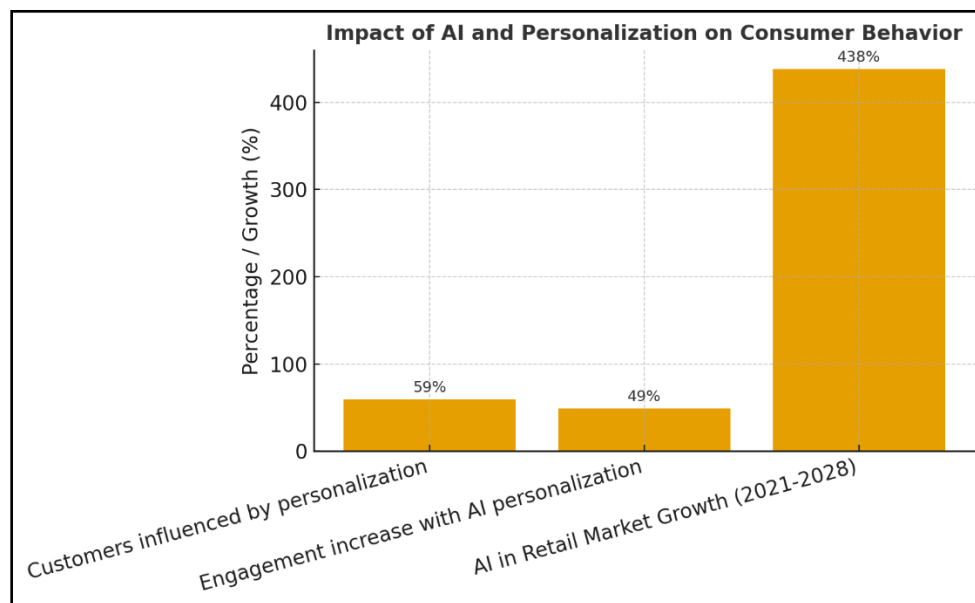


Figure 2.2.1: Impact of AI and Personalization on Consumer Behaviour (PwC, Statista, 2022).

As shown in the above graph, 59% of customers confirm that personalization directly influences their purchase decisions, and AI-driven strategies have been shown to boost engagement by up to 49%. Furthermore, the global AI in retail market is projected to grow by over 430% between 2021 and 2028, highlighting not only the dominance of recommendation systems but also the pace at which they are being adopted worldwide. While these trends emphasize the importance of AI-driven approaches, they also underline the gap for solutions that are transparent, lightweight, and adaptable - precisely the qualities *OptiPick* emphasizes.

2. Academic Research and Prototypes:

Parallel to industrial applications, academic research has investigated more structured and lightweight decision-support methods.

- **Scoring-based models:** Many prototypes rely on multi-criteria decision-making (MCDM), where users assign weights to attributes and the system produces ranked recommendations. While transparent and easy to interpret, these systems are often limited to experimental settings and lack usability features such as data storage, visualization, or export.
- **Hybrid recommenders in research:** Some academic efforts attempt to integrate the clarity of rule-based methods with the adaptability of machine learning. These systems show promise in balancing explainability with personalization but remain largely experimental, often lacking the robustness needed for real-world use.
- **NLP-powered recommenders:** More recent studies explore natural language processing to allow conversational queries (e.g., “*Find me a budget laptop with good battery life*”). While innovative, such systems require advanced infrastructure and datasets that limit their applicability in lightweight or offline-first environments.

Together, these strands of academic work reinforce the value of transparency and usability but shows that most prototypes remain incomplete or impractical for everyday users.

3. Identified Gaps:

From both industrial and academic literature, several gaps emerge:

- A shortage of offline-capable systems that can function without continuous internet connectivity.
- Limited attention to explainability, as most AI-driven systems operate as black boxes.
- An absence of lightweight tools that combine scoring transparency with essential usability features.
- Minimal exploration of frameworks that bridge rule-based clarity with AI adaptability in a scalable and practical manner.

4. Position of *OptiPick*:

In comparison to prior work, *OptiPick* addresses these gaps directly. Unlike data-heavy industrial systems, it offers an offline-first design suited for environments with limited connectivity. Unlike most academic prototypes, it goes beyond theory by including data export, session continuity, and

visualization features that improve usability. Most importantly, it prioritizes clarity and transparency through a scoring-based algorithm while also maintaining a roadmap to evolve into an AI-powered system with machine learning, NLP, and predictive analytics in the future.

Thus, while existing literature demonstrates both the power of AI recommenders and the simplicity of rule-based models, *OptiPick* positions itself as a practical bridge between the two worlds. It ensures immediate applicability through transparency and lightweight design, while keeping pace with the long-term potential of AI-driven innovation.

3. METHODOLOGY

3.1 Development Approach:

The development of the *OptiPick – A Smart Purchase Advisor* was carried out using an **Agile and Iterative Development Methodology**. This approach was selected because the system had multiple functional components such as product entry, scoring, recommendation, file handling, and statistical visualization, each of which could be developed and tested independently. Agile allowed for dividing the project into small, manageable sprints, where new features were implemented, reviewed, and refined before moving on to the next stage.

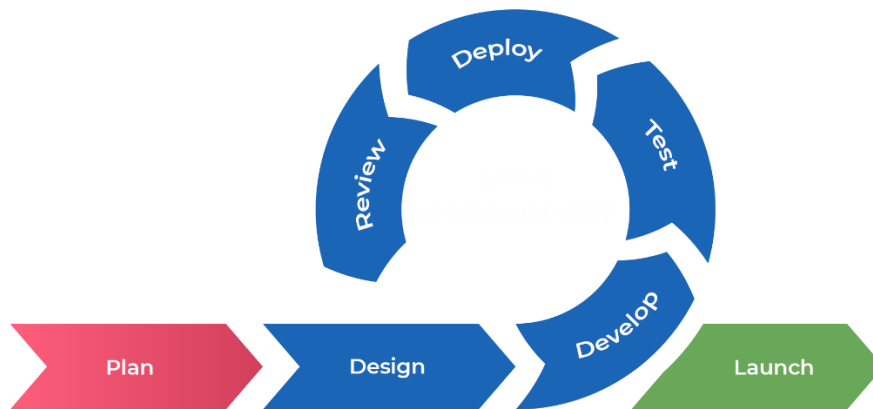


Figure 3.1.1- Agile Development Methodology

At the initial stage, the focus was on building the core functionality, such as creating the user interface with Tkinter, implementing the ability to add products, and storing essential attributes like name, price, and selected features. Once this foundation was stable, additional functionality was integrated in subsequent iterations. Examples include:

- **Iteration 1:** Setting up product data structures and input validation.
- **Iteration 2:** Adding recommendation logic using a scoring algorithm based on cost efficiency, quality, and other weighted features.
- **Iteration 3:** Enabling data persistence through JSON saving/loading and CSV export for reports.
- **Iteration 4:** Enhancing the system with data visualizations using Matplotlib and Seaborn, such as feature comparison charts, budget utilization, and confidence gauges.

- **Iteration 5:** Refining the user interface for usability, including styled tabs, buttons, and integrated images/banners.

Testing was performed at the end of each iteration to ensure that both existing and newly added functionalities worked seamlessly. This minimized the chances of error propagation and reduced debugging time. Feedback from each cycle was used to refine features and adjust priorities, ensuring that the application evolved into a complete, user-friendly tool.

The Agile approach also provided the flexibility to experiment with different scoring strategies and visualizations without disrupting the main workflow. This adaptability was crucial, since feature weighting, budget handling, and statistical representation required continuous fine-tuning.

In summary, the iterative and Agile development methodology enabled continuous improvement, modular integration of new features, effective debugging, and flexibility to adapt to new requirements, ultimately resulting in a robust and interactive product recommendation system.

3.2 Tools and Technologies Used:

The development of *OptiPick* utilized a combination of programming languages, libraries, and tools to ensure functionality, usability, and visualization. The following tools, libraries, and technologies were utilized during the development of this application:

1. Programming Language:

- a. Python (primary language for development)

2. GUI Development:

- a. Tkinter (for the desktop-based graphical user interface)
- b. ttk (for styled widgets like Treeview, Notebook tabs, etc.)

3. Data Handling & Storage:

- a. JSON (for saving and loading product lists with budget and scores)
- b. CSV (for exporting recommendations and tabular data)
- c. pandas (for structured data manipulation and analysis)

4. Visualization & Analytics:

- a. Matplotlib (for charts and plots inside Tkinter)
- b. Seaborn (for enhanced and aesthetic visualizations)

5. Image & Media Support:

- a. Pillow (PIL) library for handling images, icons, and banners

6. Other Tools & Configurations:

- Custom scoring algorithm with weighted features
- Notebook-style tab interface for smooth navigation
- File dialogs for flexible file saving and loading
- Zoomed application window and custom styling for better user experience

4. SYSTEM DESIGN

4.1 Architecture:

The *OptiPick* application is designed as a standalone desktop-based decision support system using Python Tkinter as the GUI framework. It follows a **modular layered structure** that separates core functions (data handling, scoring, recommendations) from the presentation (UI).

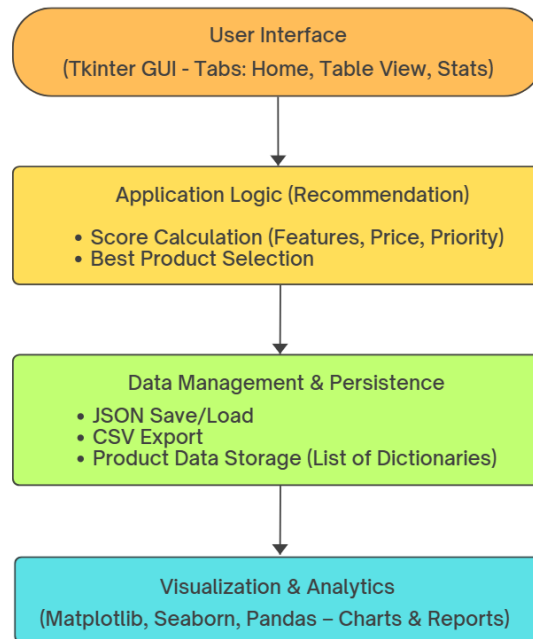


Figure 4.1.1- System Architecture

- **User Interface Layer (Presentation Layer):**
The User Interface Layer is built using **Tkinter** and designed with multiple tabs through the Notebook widget. It provides a clean and interactive platform where users can enter product details, set their budget, and select features. This layer also displays the product listing table and visualization panels. Its main role is to handle all user interactions such as adding products, updating or deleting entries, exporting results, and navigating between different views.
- **Application Logic Layer (Business Layer):**
The Application Logic Layer contains the **core recommendation mechanism** of *OptiPick*. It evaluates each product by calculating a score derived from three key factors: how well the price fits within the budget, the weighted importance of selected features, and the priority level assigned by the user. This layer manages the overall decision-making process and ensures the most suitable product is recommended. It also maintains session-related data, including the list of products and the last generated recommendation.

- **Data Management Layer:**

The Data Management Layer is responsible for saving and retrieving product data. It supports storing product lists in **JSON format** and allows users to export recommendations into **CSV files** for external use. This layer ensures data persistence between sessions and integrates with analytical tools like Pandas, Seaborn, and Matplotlib to structure the information for further visualization and reporting. This architecture ensures scalability, maintainability, and user-friendly interaction without the need for external servers or databases. Thus, ensuring that each part of the system can be developed, tested, and updated independently while maintaining smooth integration across the application.

- **Visualization and Reporting Layer:**

The Visualization and Reporting Layer provides the user with clear **analytical insights** into product comparisons. It generates visual outputs such as comparative bar charts, budget-to-recommendation fit pie charts, and a confidence gauge that reflects the reliability of the recommendation. Alongside charts, it also presents a summary table of top products with their prices and scores, making the decision-making process easier and more transparent.

Together, these layers make *OptiPick* a well-structured system that efficiently manages data, delivers accurate recommendations, and presents results in a user-friendly and visually clear manner.

4.2 Diagrams:

1. Use Case Diagram:

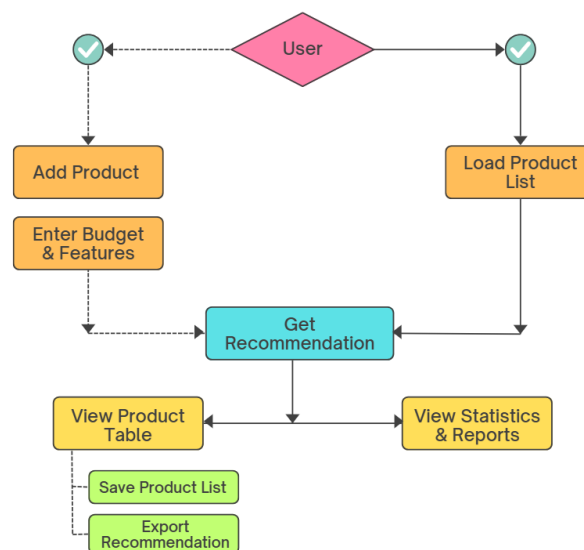


Figure 4.2.1- UML Use Case

The Use Case Diagram represents the interaction between the user and the system. It shows how the user can enter product details along with the budget and features, after which the system generates recommendations. From there, the user has the option to view the product table, save or load lists, and export results. Additionally, once recommendations are available, the user can access the statistics tab to analyze comparisons through visual insights. This diagram effectively captures the various functionalities available to the user in *OptiPick*.

2. Flowchart Diagram:

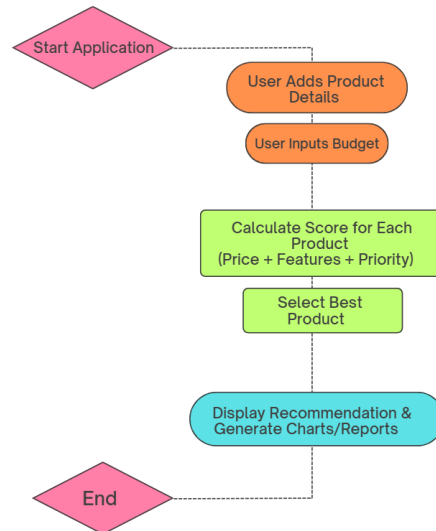


Figure 4.2.2- Recommendation Process

The Flowchart illustrates the step-by-step process of generating a product recommendation. It begins with the user adding product details and entering their budget. The system then calculates scores for each product based on price fit, feature weights, and priority levels. The best product is selected and shown as the recommendation. Afterwards, the user may view the product table, save or export data, or explore the statistical reports. This diagram clearly maps the logical workflow of the application, making the recommendation process easy to understand.

Overall, the diagrams collectively illustrate how *OptiPick* is structured, how users interact with it, and how the recommendation process flows from input to final results.

5. IMPLEMENTATION

5.1 Code Structure:

The code of *OptiPick* is modular and layered, with separate parts for the user interface, business logic, data management, and visualization. This structure ensures readability, easy maintenance, and smooth integration of features.

The Full *OptiPick* Program:

```
import tkinter as tk
from tkinter import ttk, messagebox, filedialog
import csv
import json
import matplotlib
matplotlib.use("TkAgg")
from matplotlib.backends.backend_tkagg import FigureCanvasTkAgg
import matplotlib.pyplot as plt
import seaborn as sns
import pandas as pd
from PIL import Image, ImageTk # Pillow library

# Global Variables
product_data = []
user_budget = 0.0
last_recommendation = None

feature_weights = {
    "Cost Efficiency": 7,
    "Quality": 10,
    "Durability": 8,
    "User Experience": 6,
    "Support & Maintenance": 5,
    "Popularity / Trust": 6,
    "Long-Term Value": 7,
    "Customizability": 4
}

# Functions
def get_selected_features():
    return ", ".join([feat for feat, var in feature_vars.items() if var.get()])

def safe_float(x, default=0.0):
    try:
        return float(x)
    except Exception:
        return default
```

```

def add_product():
    name = entry_name.get().strip()
    price_text = entry_price.get().strip()
    features = get_selected_features()
    priority = entry_priority.get().strip()
    if not name or not price_text or not features:
        messagebox.showwarning("Input Error", "Please fill product name, price
and select at least
                                one feature.")
    return
    try:
        price = float(price_text)
    except ValueError:
        messagebox.showwarning("Input Error", "Price must be a valid number.")
        return
    product_data.append({
        'Name': name,
        'Price': price,
        'Features': features,
        'Score': 0,
        'Priority': priority if priority else "3"
    })
    entry_name.delete(0, tk.END)
    entry_price.delete(0, tk.END)
    entry_priority.delete(0, tk.END)
    for var in feature_vars.values():
        var.set(False)
    update_table()
    messagebox.showinfo("Success", "Product added successfully.")

def update_table():
    for r in table.get_children():
        table.delete(r)
    for item in product_data:
        table.insert("", "end", values=(item['Name'], item['Price'],
item['Features'],
                                item['Priority'], item['Score']))

def delete_selected():
    sel = table.selection()
    if not sel:
        messagebox.showwarning("Selection Error", "Please select a row to
delete.")
    return
    if not messagebox.askyesno("Confirm Delete", "Delete selected product(s)?"):
        return
    for s in sel:
        vals = table.item(s, 'values')
        table.delete(s)
        # Remove from product_data (first matching name+price)
        for i, item in enumerate(product_data):
            if item['Name'] == vals[0] and str(item['Price']) == str(vals[1]):
                del product_data[i]
                break
    update_table()

```

```

def recommend_product(go_to_stats=False):
    global last_recommendation, user_budget
    if not product_data:
        messagebox.showinfo("No Data", "Please add products first.")
        return
    try:
        budget = float(entry_budget.get())
        user_budget = budget
    except ValueError:
        messagebox.showwarning("Input Error", "Please enter a valid budget.")
        return
    best = None
    max_score = -float("inf")
    for item in product_data:
        score = 0
        price = item["Price"]
        features = [f.strip() for f in item['Features'].split(",") if f.strip()]

        # Price effect
        if user_budget == 0:
            # neutral, avoid division by zero: penalize heavily if price > 0 and
            budget is 0
            if price > 0:
                score -= 50
            else:
                score += 5
        else:
            if price > user_budget:
                over_percent = (price - user_budget) / user_budget
                score -= round(over_percent * 10)
            else:
                score += 5
        # Feature weights
        for f in features:
            score += feature_weights.get(f, 5)
        # Priority
        try:
            priority_level = int(item.get('Priority', 3))
        except Exception:
            priority_level = 3
        score += priority_level
        item['Score'] = score
        if score > max_score:
            max_score = score
            best = item
    update_table()
    if best:
        last_recommendation = best
        result_label.config(text=f"Best Recommendation:\n{best['Name']}\n(₹{best['Price']})\nFeatures: {best['Features']}\nScore: {best['Score']}", fg="#2E7D32")
    else:
        result_label.config(text="No suitable product found.", fg="#D32F2F")
    if go_to_stats:

```

```

        draw_statistics()
        notebook.select(frame_stats)

def export_recommendation_csv():
    if not product_data or not last_recommendation:
        messagebox.showwarning("Missing Data", "Please add products and get a
recommendation first.")
        return
    path = filedialog.asksaveasfilename(defaultextension=".csv",
filetypes=[("CSV files", "*.csv")])
    if not path:
        return
    try:
        with open(path, 'w', newline='', encoding='utf-8') as f:
            writer = csv.writer(f)
            writer.writerow(["Name", "Price", "Features", "Priority", "Score"])
            for p in product_data:
                writer.writerow([p['Name'], p['Price'], p['Features'],
p['Priority'], p['Score']])
            writer.writerow([])
            writer.writerow([f"Recommended Product:
{last_recommendation['Name']}"])
            messagebox.showinfo("Exported", f"Saved CSV to {path}")
    except Exception as e:
        messagebox.showerror("Export Error", str(e))

def save_product_list():
    if not product_data:
        messagebox.showwarning("No Data", "There are no products to save.")
        return
    path = filedialog.asksaveasfilename(defaultextension=".json",
filetypes=[("JSON
files", "*.json")])
    if not path:
        return
    try:
        data = {"budget": user_budget, "products": product_data}
        with open(path, 'w', encoding='utf-8') as f:
            json.dump(data, f, indent=2)
            messagebox.showinfo("Saved", "Product list saved successfully.")
    except Exception as e:
        messagebox.showerror("Save Error", str(e))

def load_product_list():
    global product_data, user_budget, last_recommendation
    path = filedialog.askopenfilename(defaultextension=".json",
filetypes=[("JSON files", "*.json")])
    if not path:
        return
    try:
        with open(path, 'r', encoding='utf-8') as f:
            loaded = json.load(f)
            product_data = loaded.get("products", [])
            user_budget = loaded.get("budget", 0.0)
            entry_budget.delete(0, tk.END)

```

```

        entry_budget.insert(0, str(user_budget))
        update_table()
        # run recommendation automatically after loading (but not auto-switch)
        recommend_product(go_to_stats=False)
        messagebox.showinfo("Loaded", "Product list loaded successfully.")
    except Exception as e:
        messagebox.showerror("Load Error", str(e))

def draw_statistics():
    # clear frame
    for widget in frame_stats.wininfo_children():
        widget.destroy()
    if not product_data or not last_recommendation:
        tk.Label(frame_stats, text="No recommendation available. Add products
and get recommendation
        first.", font=("Helvetica", 14, "bold"), fg="#D32F2F",
bg=frame_bg).grid(row=0, column=0,
        columnspan=2, pady=20)
        return
    if len(product_data) < 2:
        tk.Label(frame_stats, text="Not enough products to compare (need at
least 2).",
        font=("Helvetica", 14), fg="#D32F2F", bg=frame_bg).grid(row=0,
column=0,
        columnspan=2, pady=20)
        return
    sns.set_style("whitegrid")
    # Product feature comparison (DataFrame)
    feature_names = sorted(feature_weights.keys())
    df_comp = []
    for p in product_data[:5]: # first 5 products
        feats = [f.strip() for f in p['Features'].split(",") if f.strip()]
        for f in feature_names:
            score = feature_weights.get(f, 0) if f in feats else 0
            df_comp.append({"Product": p["Name"], "Feature": f, "Score": score})
    df_comp = pd.DataFrame(df_comp)
    # Grouped Bar Chart (features x products)
    fig1, ax1 = plt.subplots(figsize=(10, 4))
    sns.barplot(data=df_comp, x="Feature", y="Score", hue="Product", ax=ax1,
palette="Set2")
    ax1.set_title("Products Feature Score Comparison")
    ax1.set_ylabel("Score")
    plt.setp(ax1.get_xticklabels(), rotation=45, ha="right")
    fig1.tight_layout() # automatically adjust margins
    canvas1 = FigureCanvasTkAgg(fig1, master=frame_stats)
    canvas1.draw()
    canvas1.get_tk_widget().grid(row=1, column=0, columnspan=2, sticky="nsew",
padx=20, pady=10)
    # Budget vs Recommendation pie
    fig2, ax2 = plt.subplots(figsize=(3,3), subplot_kw={'aspect':'equal'})
    if user_budget == 0:
        percent_fit = 0.0
    else:
        percent_fit = min(last_recommendation["Price"] / user_budget, 1.5)
    used = min(percent_fit, 1.0)

```

```

        remaining = max(1.0 - used, 0.0)
        ax2.pie([used, remaining],
                labels=[f"Used ({used*100:.1f}%)", f"Remaining
({remaining*100:.1f}%)"],
                startangle=90, colors=['#4CAF50', '#E0E0E0'])
        ax2.set_title(f"Budget Fit (Based on ₹{user_budget:.2f})")
        canvas2 = FigureCanvasTkAgg(fig2, master=frame_stats)
        canvas2.draw()
        canvas2.get_tk_widget().grid(row=2, column=0, sticky="nsew", padx=10,
pady=10)
        # Confidence gauge
        fig3, ax3 = plt.subplots(figsize=(3,3))
        confidence = max(0.0, min(last_recommendation.get("Score", 0) / 100.0, 1.0))
        ax3.pie([confidence, 1-confidence], colors=['#0288D1', '#E0E0E0'],
startangle=90)
        centre = plt.Circle((0,0), 0.7, fc='white')
        fig3.gca().add_artist(centre)
        ax3.set_title(f"Recommendation Confidence: {confidence*100:.1f}%")

        canvas3 = FigureCanvasTkAgg(fig3, master=frame_stats)
        canvas3.draw()
        canvas3.get_tk_widget().grid(row=2, column=1, sticky="nsew", padx=10,
pady=10)
        # Table summary
        label_table = tk.Label(frame_stats, text="Top Products Overview",
font=("Helvetica", 14),
                                bg=frame_bg, fg="black")
        label_table.grid(row=0, column=2, pady=5, padx=10)
        top_sorted = sorted(product_data, key=lambda x: x['Score'],
reverse=True)[:5]
        cols = ("Name", "Price", "Score")
        tree = ttk.Treeview(frame_stats, columns=cols, show='headings', height=5)
        for c in cols:
            tree.heading(c, text=c)
            tree.column(c, width=120)
        for p in top_sorted:
            tree.insert("", "end", values=(p['Name'], f"₹{p['Price']:.2f}",
p['Score']))
        tree.grid(row=1, column=2, pady=10, padx=10, sticky="nsew")

# UI
root = tk.Tk()
root.title("OptiPick - Optimal Product Recommender")
root.state('zoomed')
frame_bg = "#F5F5F5"
root.configure(bg=frame_bg)

# Favicon Conversion and Application
# logo_img = Image.open("AI_OptiPick_Logo.png")
# logo_img.save("favicon.ico", format="ICO", sizes=[(64,64)])
root.iconbitmap("favicon.ico")

# Style
style = ttk.Style()
style.theme_use('clam')

```

```

style.configure("TNotebook", background=frame_bg)
style.configure("TNotebook.Tab", font=("Helvetica", 12, "bold"), padding=[10,6])
style.map("TNotebook.Tab",
        background=[("selected", "#4CAF50"), ("!selected", "#E0E0E0")],
        foreground=[("selected", "white"), ("!selected", "#333333")])
style.configure("Treeview", font=("Helvetica", 10), rowheight=24)
style.configure("Treeview.Heading", font=("Helvetica", 11, "bold"))

# Notebook and tabs
notebook = ttk.Notebook(root)
notebook.pack(fill="both", expand=True, padx=8, pady=8)

frame_add = tk.Frame(notebook, bg=frame_bg)
frame_table = tk.Frame(notebook, bg=frame_bg)
frame_stats = tk.Frame(notebook, bg=frame_bg)

notebook.add(frame_add, text="Home")
notebook.add(frame_table, text="Table View")
notebook.add(frame_stats, text="Statistics")

# To refresh statistics tab upon any tab changed event
def on_tab_changed(event):
    # Check if current tab is Statistics (3rd tab-index 2)
    if notebook.index(notebook.select()) == 2:
        draw_statistics()

# Bind tab switch to the function
notebook.bind("<<NotebookTabChanged>>", on_tab_changed)

# Frame 1: Home / Add Product UI
tk.Label(frame_add, text="OptiPick", font=("Helvetica", 20, "bold"),
bg=frame_bg, fg="#FF3131").place(x=500, y=50)
tk.Label(frame_add, text="Your own Smart Product Advisor", font=("Helvetica",
18), bg=frame_bg, fg="#FF5757").place(x=450, y=83)
tk.Label(frame_add, text="Add New Product", font=("Helvetica", 18, "bold"),
bg=frame_bg, fg="#0288D1").place(x=200, y=120)

entry_name = tk.Entry(frame_add, font=("Helvetica", 14), width=40, bd=2,
relief="groove")
entry_price = tk.Entry(frame_add, font=("Helvetica", 14), width=40, bd=2,
relief="groove")
entry_budget = tk.Entry(frame_add, font=("Helvetica", 14), width=40, bd=2,
relief="groove")
entry_priority = tk.Entry(frame_add, font=("Helvetica", 14), width=40, bd=2,
relief="groove")
labels = ["Product Name: ", "Price (₹): ", "Select Features: "]
for i, label in enumerate(labels):
    tk.Label(frame_add, text=label, font=("Helvetica", 14, "bold"), bg=frame_bg,
fg="#333333").place(x=200, y=190 + i*60)
tk.Label(frame_add, text="Your Budget (₹): ", font=("Helvetica", 14, "bold"),
bg=frame_bg, fg="#333333").place(x=200, y=500)
tk.Label(frame_add, text="Priority Level (1-5): ", font=("Helvetica", 14,
"bold"), bg=frame_bg, fg="#333333").place(x=200, y=550)
entry_name.place(x=420, y=190)
entry_price.place(x=420, y=249)

```

```

entry_budget.place(x=420, y=500)
entry_priority.place(x=420, y=550)
feature_vars = {}
feature_frame = tk.Frame(frame_add, bg=frame_bg)
feature_frame.place(x=410, y=308)
for i, feat in enumerate(feature_weights):
    var = tk.BooleanVar()
    chk = tk.Checkbutton(feature_frame, text=feat, variable=var,
font=("Helvetica", 12), bg=frame_bg, fg="#333333", selectcolor="#BBDEFB")
    chk.grid(row=i//2, column=i%2, sticky="w", padx=10, pady=5)
    feature_vars[feat] = var
btn_add = tk.Button(frame_add, text="Add Product", font=("Helvetica", 12,
"bold"), bg="#4CAF50", fg="white", bd=0, padx= 5, pady=5, command=add_product)
btn_reco = tk.Button(frame_add, text="Get Recommendation", font=("Helvetica",
12, "bold"), bg="#0288D1", fg="white", bd=0, padx= 5, pady=5, command=lambda:
recommend_product(go_to_stats=False))
btn_export = tk.Button(frame_add, text="Export to CSV", font=("Helvetica", 12,
"bold"), bg="#FBC02D", fg="white", bd=0, padx= 5, pady=5,
command=export_recommendation_csv)
btn_view_stats = tk.Button(frame_add, text="Recommend & View Statistics",
font=("Helvetica", 12, "bold"), bg="#7B1FA2", fg="white", bd=0, relief="flat",
padx= 5, pady=5, command=lambda: recommend_product(go_to_stats=True))
btn_add.place(x=950, y=249)
btn_reco.place(x=950, y=299)
btn_export.place(x=950, y=349)
btn_view_stats.place(x=950, y=399)
canvas_bg = tk.Canvas(frame_add, width=2000, height=500, bg="#FFDE59",
highlightthickness=0)
canvas_bg.place(x=0, y=640)
canvas_bg.create_rectangle(10, 10, 140, 140, fill="#FFDE59", outline="#FFDE59",
width=2)
result_label = tk.Label(frame_add, text="No Recommendation yet..",
font=("Helvetica", 14), bg="#FFDE59", wraplength=700, justify="left")
result_label.place(x=420, y=700)
canvas_img = tk.Canvas(frame_add, width=300, height=640, bg=frame_bg,
highlightthickness=0)
canvas_img.place(x=1250, y=0)
image1 = Image.open("OptiPick_sideBanner.png")
photo = ImageTk.PhotoImage(image1)
canvas_img.create_image(0, 0, image=photo, anchor="nw")
canvas_img.photo = photo
canvas_img = tk.Canvas(frame_add, width=300, height=640, bg=frame_bg,
highlightthickness=0)
canvas_img.place(x=1250, y=650)
image2 = Image.open("OptiPick_sideBanner2.png")
photo = ImageTk.PhotoImage(image2)
canvas_img.create_image(0, 0, image=photo, anchor="nw")
canvas_img.photo = photo

# Frame 2: Table View
tk.Label(frame_table, text="Product Table", font=("Helvetica", 16, "bold"),
bg=frame_bg, fg="#0288D1").pack(pady=12)
table_frame = tk.Frame(frame_table, bg=frame_bg)
table_frame.pack(fill="both", expand=True, padx=24, pady=8)
cols = ("Name", "Price", "Features", "Priority", "Score")

```

```

table = ttk.Treeview(table_frame, columns=cols, show="headings",
style="Treeview")
for col in cols:
    table.heading(col, text=col)
    table.column(col, width=160 if col != "Score" else 90, anchor="w")
table.pack(fill="both", expand=True)

table_btn_frame = tk.Frame(frame_table, bg=frame_bg)
table_btn_frame.pack(pady=12)
tk.Button(table_btn_frame, text="Delete Product", font=("Helvetica", 11,
"bold"), bg="#D32F2F", fg="white", bd=0, relief="flat",
command=delete_selected).pack(side="left", padx=6)
tk.Button(table_btn_frame, text="Save Product List", font=("Helvetica", 11,
"bold"), bg="#0288D1", fg="white", bd=0, relief="flat",
command=save_product_list).pack(side="left", padx=6)
tk.Button(table_btn_frame, text="Load Product List", font=("Helvetica", 11,
"bold"), bg="#FBC02D", fg="white", bd=0, relief="flat",
command=load_product_list).pack(side="left", padx=6)
tk.Button(table_btn_frame, text="Refresh Table", font=("Helvetica", 11, "bold"),
bg="#607D8B", fg="white", bd=0, relief="flat",
command=update_table).pack(side="left", padx=6)

# Initial state
notebook.select(frame_add)
root.mainloop()

```

Summary of Code Components:

1. Library Imports:

At the very beginning, the program imports essential libraries: tkinter for GUI, ttk, messagebox, and filedialog for interface and user interaction, csv/json for data storage, matplotlib and seaborn for plotting, pandas for tabular handling, and PIL for image display.

2. Global Variables:

Right after imports, variables like product_data, user_budget, last_recommendation, and a feature_weights dictionary are defined to hold product details, user budget, and scoring logic.

3. Core Functions:

The middle section of the code defines all the logic-based functions. These are defined first so they can be called later in the GUI.

○ Data Handling –

add_product() → Adds product details (name, price, features, priority) into the dataset.

delete_selected() → Deletes chosen products from the table and dataset.

update_table() → Refreshes the product table view.

save_product_list() & load_product_list() → Save/load product data in JSON format.

- **Recommendation Logic –**

`get_selected_features()` → Collects selected product features.

`safe_float()` → Safely converts values to float.

`recommend_product()` → Calculates scores, compares with budget, and selects the best product.

Logic Explanation:

- **Budget Check:** Each product's price is compared with the user's budget. Products within budget gain points, while products over budget lose points.
- **Feature Weights:** Every selected feature (e.g., Quality, Durability, Cost Efficiency) adds predefined weight to the product's score.
- **Priority Level:** User-assigned priority (1–5) adds extra points, giving importance to urgent or preferred products.
- **Final Scoring:** The total score is calculated by combining budget effect, feature weights, and priority.
- **Best Product Selection:** The product with the **highest score** is chosen as the final recommendation and displayed to the user.

In short, the logic finds the most balanced product by combining affordability, features, and user preference.

- **Data Export –**

`export_recommendation_csv()` → Exports product list and recommendation to CSV.

- **Visualization –**

`draw_statistics()` → Creates feature comparison bar chart, budget fit pie chart, confidence gauge, and a summary table.

4. Graphical User Interface (Structured After Functions):

After defining logic functions, the GUI is set up:

- **Main Window:** Created using `tk.Tk()` with title, styling, and favicon.
- **Notebook Tabs:** Three tabs (Home, Table View, Statistics) are created using `ttk.Notebook()`.
- **Home Tab:** Contains input fields (name, price, budget, priority), feature checkboxes, and buttons (Add Product, Get Recommendation, Export, View Statistics) along with banners and labels.
- **Table View Tab:** Shows product data in a Treeview table with buttons for Delete, Save, Load, and Refresh.
- **Statistics Tab:** Dynamically updated with feature comparison charts, budget pie chart, confidence gauge, and summary table.

5. Event Handling (At the End of GUI Section):

Functions like `on_tab_changed()` are bound to events (`<<NotebookTabChanged>>`) to refresh statistics when the Statistics tab is selected.

6. Program Flow (Final Part):

At the very end, the program selects the Home tab as the default starting point and calls `root.mainloop()` to continuously run the application.

In short, the structure is layered: imports first → global variables → function definitions → GUI construction → event handling → program execution. This makes the program modular, easy to maintain, and ensures that all GUI components can call the predefined functions smoothly.

5.2 Key Features:

Key features of *OptiPick* include:

- **Product Input & Management** – Add, update, delete, and view products with details such as price, budget, and features.
- **Recommendation Engine** – Calculates product scores based on budget fit, feature importance, and priority levels.
- **Data Handling** – Save and load product lists in JSON format, and export recommendations to CSV files.
- **Statistical Insights** – View comparative bar charts, budget vs. recommendation pie charts, and a confidence gauge.
- **Session Continuity** – Automatically retains the last recommendation for user convenience.

In essence, these features work together to provide users with a structured, efficient, and visually guided way of selecting the most suitable product.

6. DEPLOYMENT

6.1 Deployment Process:

The deployment of *OptiPick* follows a straightforward process, as the software is designed to run locally without requiring complex server infrastructure. The program is packaged as a standalone Python application that can be executed on any system with Python installed. The complete project folder, including the Python source code, JSON/CSV sample files, and required libraries, was uploaded to a **GitHub repository**. This makes the application publicly available and easy to download for anyone with a basic Python setup. Users can clone or download the repository and run the program directly, as the necessary libraries (Tkinter, Pandas, Matplotlib, and Seaborn) are widely available in most environments. By using GitHub for deployment, *OptiPick* remains version-controlled, publicly accessible, and easy to update or extend in the future.

To ensure *OptiPick* reaches a wider audience, the project is shared through its public GitHub repository, making it easily accessible to anyone online. The repository link can be promoted across **educational forums, open-source communities, and student project platforms**, allowing more users to discover and adopt the software. Since *OptiPick* is designed for efficient product comparison, it is particularly useful for its target audience- **students, researchers, and budget-conscious buyers**—who can benefit from its features without additional cost. Furthermore, by keeping the repository open, the project invites contributions from developers and testers, enabling continuous improvement and wider adoption through community involvement.

6.2 User Documentation:

OptiPick is supported with a **User Guide** that explains installation and usage in a step-by-step manner. The documentation provides instructions on how to download the project from GitHub, open the program, and navigate through its different tabs. It includes guidance on adding products, entering budgets, generating recommendations, and viewing product tables. Users are also shown how to save and load product lists, export recommendations to CSV files, and interpret the statistical charts. This documentation ensures that both technical and non-technical users can easily operate the software.

Installation & Setup:

1. Download the Project:

- Visit the *OptiPick* GitHub repository.
- Download or clone the repository to your local machine.

2. Run the Program

- Ensure Python (3.8 or later) is installed.
- Install required libraries if missing:
`pip install tkinter pillow matplotlib seaborn pandas`
- Run the application using:
`python OptiPick.py`

Quick Start: Using *OptiPick* (Step-by-Step):

1. Open the app → Home tab

You'll see fields for Product Name, Price (₹), Your Budget (₹), Priority (1–5), and a list of feature checkboxes.

2. Enter your budget (one common budget for all products).

Type it once in **Your Budget (₹)** on the Home tab.

3. Add at least two products.

- Fill **Product Name** and **Price (₹)**.
- Select relevant **Features** (tick the checkboxes).
- Set **Priority** (Priority 5 = highest urgency/importance and Priority 1 = lowest.).
- Click **Add Product**.
- Repeat for product 2, 3, ... (minimum two products works best).

4. Get the recommendation.

Click **Get Recommendation** to see the **best product** (shown on the yellow panel): name, price, selected features, and the computed **Score**.

5. See everything in a table.

Go to the **Table View** tab (beside Home) to view all products in a grid.

- Use **Delete Product** to remove selected rows.
- Use **Save Product List** to save as **.json** in any directory.
- Use **Load Product List** to import a saved list from any directory.
- Use **Export to CSV** to generate a sharable **.csv** report by saving the product data in any directory.

6. View charts and insights.

- Click **Recommend & View Statistics** (on Home) or switch to the **Statistics** tab.
- You'll see:
 - **Feature comparison bar chart** (how products stack on chosen features)
 - **Budget fit pie chart** (how well the top pick fits your budget)
 - **Confidence gauge** (relative confidence based on the score)
 - **Top Products summary table** (name, price, score)

7. Iterate if needed.

Add/edit products, adjust features/priority, or change the budget, then click **Get Recommendation** again to refresh results and charts.

Tip: For clear results, keep product names unique and select only the features that truly matter for each product.

The User Manual documentation ensures that users can quickly understand, navigate, and make the most of *OptiPick*'s features with minimal effort. This makes *OptiPick* simple to share, use, and grow among its target audience, ensuring wider accessibility and continuous improvement.

7. RESULTS AND DISCUSSION

7.1 Achievements:

Project Achievements:

The *OptiPick* application successfully delivered a complete **product recommendation system** that combines budget management, feature prioritization, and data visualization. Users can add multiple products, assign urgency levels, and generate optimized recommendations supported by detailed charts and tables. Beyond the core recommendation logic, the project integrated several advanced features such as **JSON-based data persistence**, **CSV export for reporting**, and **visualizations using Matplotlib and Seaborn**. These include bar charts for product-feature comparison, pie/donut charts for budget allocation, and confidence gauges to measure recommendation reliability. A side-by-side **comparison window** was also introduced to assist users in evaluating multiple products simultaneously.

User Validation:

Another notable achievement was **user testing and acceptance**. The application was shared among a small group of peers and classmates, with approximately five users actively testing it. Feedback suggested that the interface was **intuitive** and the recommendation logic **convincing**. The users appreciated the ability to save and reload product lists, while others valued the statistical charts for deeper insights. Many highlighted that **visualizations embedded within the application** enhanced usability by avoiding external tools, and the CSV reports were considered helpful for further independent analysis. Overall, the feedback confirmed that *OptiPick* delivers **real value for budget-conscious users**, particularly students and researchers, in making informed purchasing decisions.

Personal Achievements:

From a personal perspective, this project offered significant technical and professional learning opportunities. I gained hands-on experience with Tkinter and ttk.Treeview for building structured GUIs, along with implementing input validation, error alerts, and deletion logic to ensure reliability. Working with CSV and JSON handling strengthened my knowledge of data persistence and structured exports, while integrating Tkinter file dialogs improved user experience. I also explored the Pillow (PIL) library for image handling, learned to convert PNG to .ICO for setting a favicon, and practiced designing UIs with all three Tkinter layout managers- `grid()`, `pack()`, `place()`.

On the data science side, I gained valuable experience in mapping user-defined feature weights into quantitative scores, thereby applying decision-making models in practice. I also experimented with fuzzy similarity matching using `difflib.SequenceMatcher`, which reflects real-world

applications of partial fuzzy logic in recommendation engines. The integration of Matplotlib and Seaborn charts into a live Tkinter application was especially rewarding, as it bridged the gap between analytics and interactive visualization. This achievement not only enhanced the system but also opened my perspective to the potential of combining GUI development with data science techniques.

7.2 Challenges Faced:

The development process of *OptiPick* involved overcoming several technical and design challenges, many of which directly shaped the final robustness of the system:

1. Interface and Layout Management

- The Treeview table initially failed to refresh correctly, causing duplicate rows instead of updates. This required implementing proper clearing and re-populating logic.
- Aligning widgets with the `grid()` layout manager was another challenge, as minor misplacements disrupted the overall flow. Repeated fine-tuning was necessary to ensure consistency across tabs.
- Integrating the side-banner image into the Tkinter layout required adjusting x-y coordinates carefully to avoid misalignment and disconnection from the UI.

2. Recommendation Logic Refinement

- In early iterations, all products scored the same since feature importance was not adequately differentiated. The scoring algorithm had to be revised multiple times to balance feature weights, price fit, and priority.
- The priority label was initially being stored incorrectly because user input was taken directly instead of being matched against defined priority levels. This was corrected to ensure accurate urgency handling.
- The recommendation loop itself needed restructuring to ensure both accuracy and traceability, especially when handling ties or products near the budget.

3. Validation and Data Handling

- Implementing **real-time validation** without interrupting the user experience was tricky. For instance, incorrect inputs had to be flagged without blocking valid entries.
- Handling JSON data loading introduced errors when products had missing or invalid keys. Safeguards were added to prevent crashes and default values were applied where necessary.
- Managing priority values and budget allocation while loading from JSON was also challenging, as inconsistencies could break recommendation calculations.

4. Data Export and Reporting

- Preserving data consistency in CSV exports required careful string formatting, especially when handling floating-point scores.
- Ensuring that the last recommendation was valid before allowing export was essential to avoid empty or inaccurate reports.
- Handling user cancellation in file dialogs had to be implemented gracefully so the program did not break the workflow.

5. Visualization Integration

- Integrating Matplotlib/Seaborn plots directly into Tkinter without opening external windows was complex. This was achieved with FigureCanvasTkAgg, but required careful refresh logic.
- The grouped bar chart's x-axis labels were often cut off due to table margins, which needed repeated fine adjustments.
- Positioning the product table in the Statistics tab alongside the visualizations required restructuring the layout for clarity.

6. Additional Technical Hurdles

- A favicon conversion error occurred while converting PNG to ICO using Pillow, which was fixed by correcting argument usage.
- Managing simultaneous use of Pandas, Matplotlib, and Seaborn created dependency conflicts at times, but these were resolved through version alignment and careful import structuring.

Through tackling these challenges, *OptiPick* evolved into a polished and reliable application. Each issue- whether in layout, logic, validation, or visualization—contributed to strengthening the overall design and usability of the project.

8. CONCLUSION

8.1 Summary of Findings:

The *OptiPick* application was successfully developed as a fully functional offline **Price Comparison and Product Recommendation System**, providing users with an intelligent and user-friendly tool to make informed purchasing decisions. It integrates budget management, feature prioritization, and scoring-based recommendations to evaluate and rank products based on user preferences and constraints. With JSON-based data persistence and dynamic CSV report generation, the system ensures smooth data handling, continuity across sessions, and easy reporting. The inclusion of visualizations using Matplotlib and Seaborn—such as bar charts for feature comparison, pie/donut charts for budget allocation, and confidence gauges for recommendation reliability—enhanced interpretability, enabling users to quickly understand product scores and make data-driven decisions.

From a **data science perspective**, this project provided substantial practical learning. I translated subjective user inputs, like product features and priority levels, into quantitative feature weights, applying a multi-criteria scoring algorithm to determine product rankings. This exercise reinforced concepts of feature engineering, normalization, weighted scoring, and aggregate evaluation, which are central to recommendation systems and decision analytics. Implementing partial fuzzy logic with `difflib.SequenceMatcher` allowed the system to handle approximate matches and partial similarities, a critical real-world challenge when dealing with user-entered data, textual comparisons, or incomplete information. These experiences bridged the gap between pure algorithmic thinking and practical analytics, highlighting the importance of structured data processing and intelligent evaluation in software design.

The project also provided extensive hands-on experience in **software development**. I gained proficiency in GUI design using Tkinter, including `tk.Treeview` for structured tabular displays, layout management (`grid()`, `pack()`, `place()`), and integrating images with Pillow for enhanced UI aesthetics. I implemented robust input validation, error handling, and row deletion mechanisms, ensuring reliability and improved user experience. Combining these elements with dynamic charts and statistics embedded directly into the GUI demonstrated how coding, data science, and visualization can be unified to create a comprehensive, interactive application.

On a **personal level**, working on *OptiPick* opened a new perspective on how data-driven techniques can enhance everyday decision-making. It strengthened my understanding of mapping user-defined preferences to numerical evaluation, creating actionable insights, and delivering a tool that is both functional and visually informative. The project also cultivated problem-solving skills, especially in managing trade-offs between functionality, usability, and analytical accuracy. Overall, this experience not only enhanced my technical and analytical skills but also inspired

further exploration into recommendation engines, predictive modeling, and interactive data visualization, highlighting the transformative potential of applying data science in real-world applications.

8.2 Future Work:

1. Interactive Data Dashboard Integration:

Future iterations could incorporate an interactive analytics dashboard within the application. This would allow users to filter products by features, visualize comparisons dynamically, and explore budget impact interactively, providing a more engaging and insightful decision-making experience entirely offline.

2. Offline Predictive Insights:

Future versions could integrate predictive analytics within the offline environment, such as estimating future product value trends, potential price drops, or durability forecasts. This would give users a forward-looking perspective, enhancing their ability to make data-driven decisions over time without relying on online connectivity.

3. Integration of Fuzzy Matching & NLP:

Expanding the fuzzy logic implementation could allow *OptiPick* to interpret natural language product descriptions or approximate queries. This would make the system more resilient to human input variations, misspellings, or descriptive entries, improving both usability and recommendation reliability in real-world scenarios.

4. Real-Time Data Integration (Optional Extension):

While the core system is offline, an optional online module could fetch live product prices or metadata from e-commerce platforms via APIs. This extension would allow *OptiPick* to provide up-to-date recommendations while preserving the standalone offline functionality for users who prefer local processing.

5. Advanced Recommendation Algorithms:

Future research could explore integrating machine learning techniques, such as collaborative filtering, regression-based scoring, or clustering algorithms. These approaches could enable personalized recommendations, learn from user interaction patterns, and provide more sophisticated insights while still maintaining modularity and offline capability when desired.

6. Enhanced User Interface for Web/Mobile:

Developing a responsive web or mobile interface could broaden accessibility and allow users to interact with the system more intuitively. Features such as drag-and-drop comparisons, interactive sliders for budget/priority adjustments, and real-time visual feedback would create a richer user experience while maintaining the simplicity and lightweight nature of *OptiPick*.

Collectively, these enhancements represent natural evolution paths for *OptiPick*, combining advanced data science techniques, interactive visualization, and user-centric design. They maintain the original philosophy of a lightweight, standalone, offline-friendly system while exploring avenues for research, predictive analytics, and enhanced user engagement.

9. REFERENCES

Books:

1. Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein – “Introduction to Algorithms” [3rd Edition]

Python Documentations:

1. Python tkinter.ttk Treeview:
<https://docs.python.org/3/library/tkinter.ttk.html#treeview>
2. Python tkinter.ttk Notebook:
<https://docs.python.org/3/library/tkinter.ttk.html#tab-options>

Online Example:

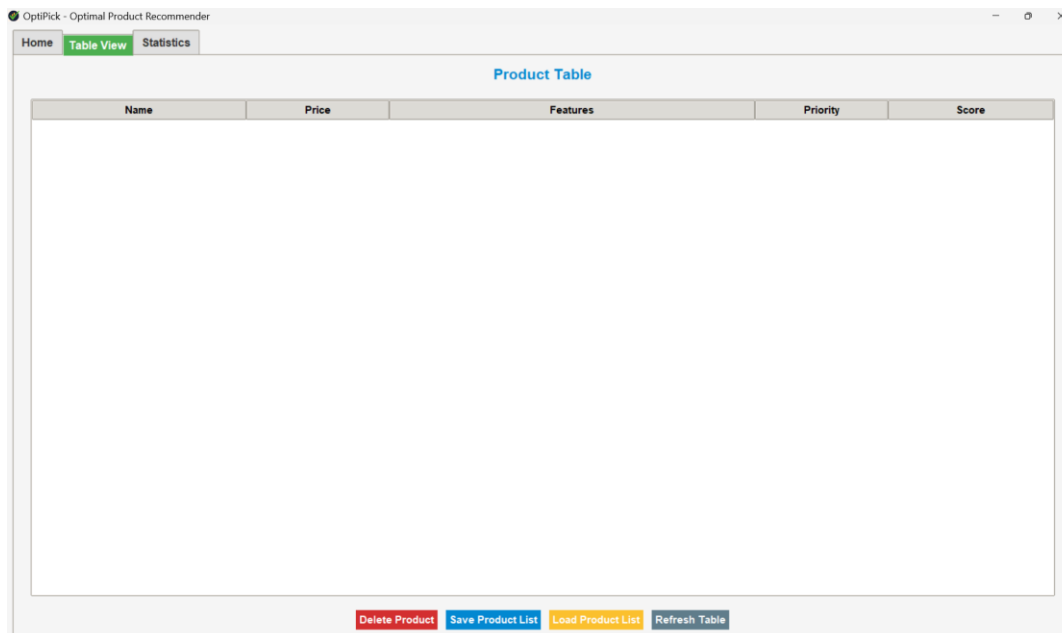
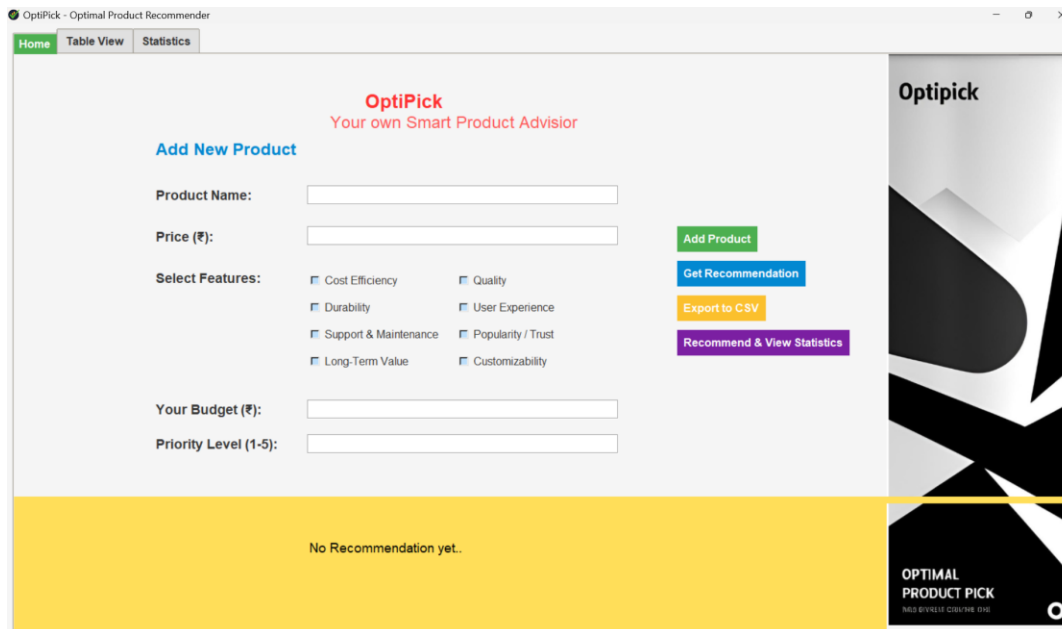
1. Seaborn visualization examples
https://seaborn.pydata.org/examples/faceted_histogram.html

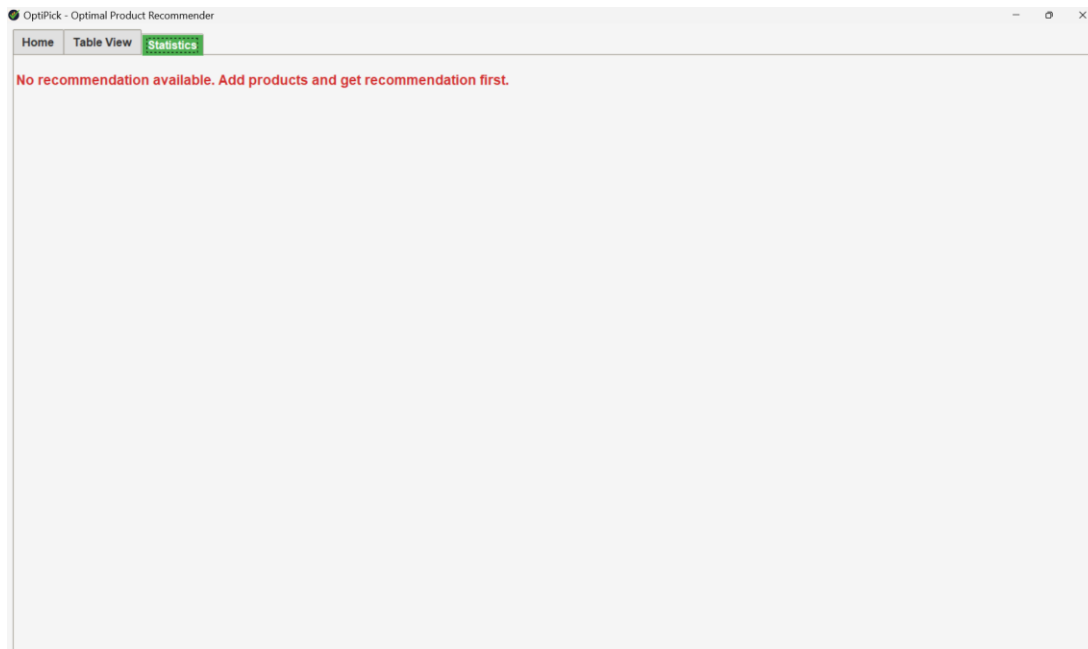
YouTube Tutorials:

1. Corey Schafer - Matplotlib Tutorial (Part 2): Bar Charts and Analyzing Data from CSVs
<https://www.youtube.com/watch?v=nKxLfUrLE8>
2. Corey Schafer - Matplotlib Tutorial (Part 1): Creating and Customizing Our First Plots
<https://www.youtube.com/watch?v=UO98IJQ3QGI>
3. Keith Galli - Complete Python Pandas Data Science Tutorial! (Reading CSV/Excel files, Sorting, Filtering, Groupby)
<https://www.youtube.com/watch?v=vmEHCJofslg>

10. APPENDICES

Appendix A: GUI Screenshots



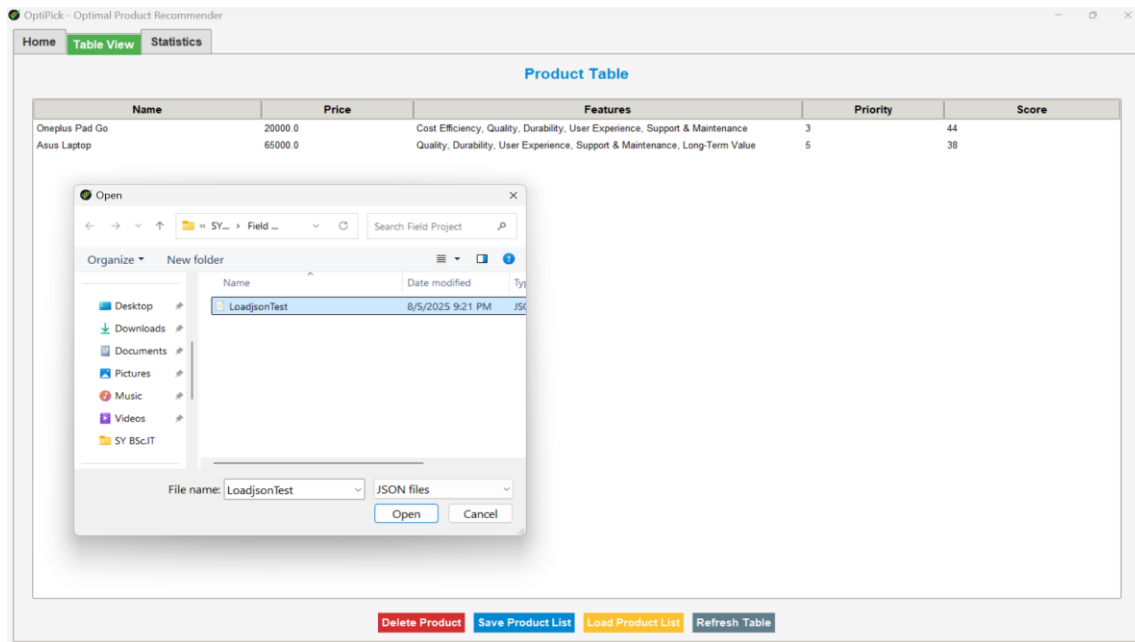


Appendix B: Sample Input and Output Data (JSON File)

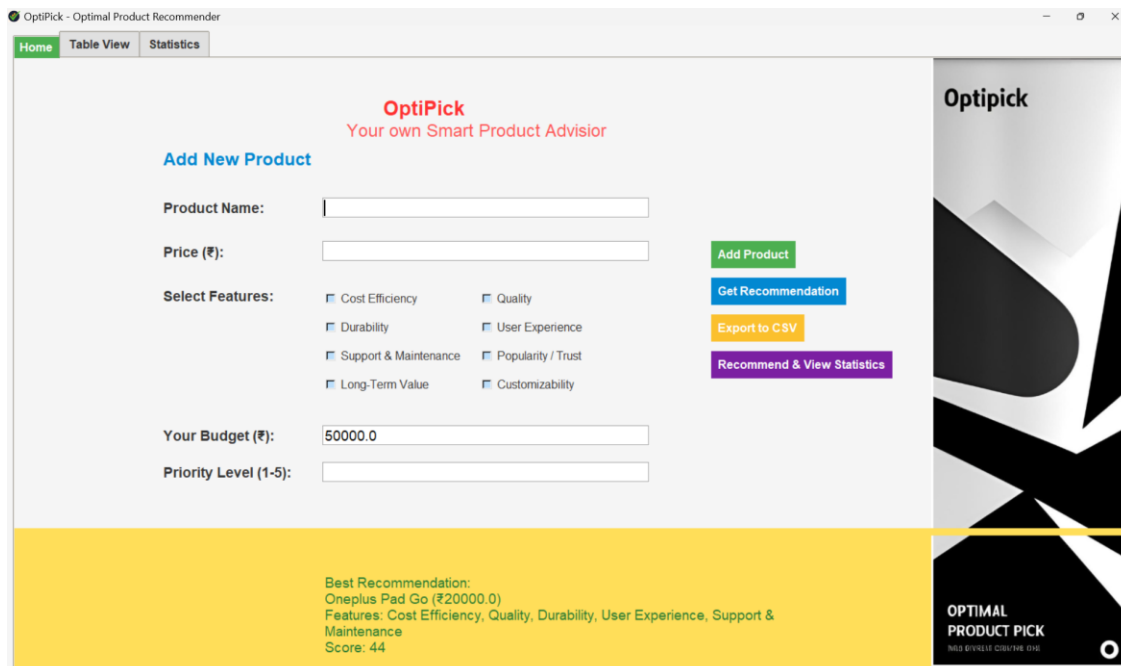
Sample JSON File:

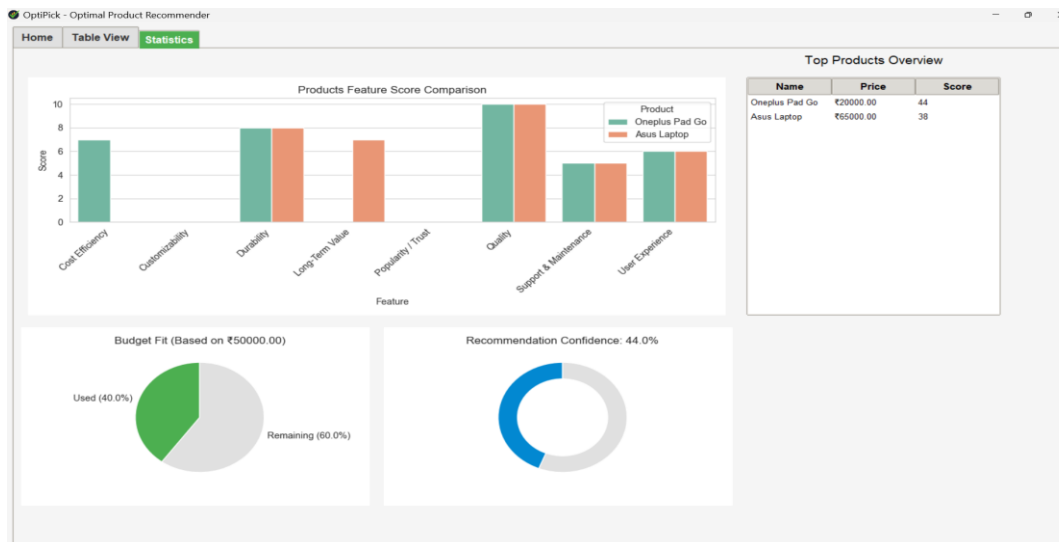
```
{ LoadjsonTest.json X
C: > SY BSc.IT > Field Project > {} LoadjsonTest.json > ...
1  {
2      "budget": 50000.0,
3      "products": [
4          {
5              "Name": "Oneplus Pad Go",
6              "Price": 20000.0,
7              "Features": "Cost Efficiency, Quality, Durability, User Experience, Support & Maintenance",
8              "Score": 49,
9              "Priority": 3
10         },
11         {
12             "Name": "Asus Laptop",
13             "Price": 65000.0,
14             "Features": "Quality, Durability, User Experience, Support & Maintenance, Long-Term Value",
15             "Score": 38,
16             "Priority": 5
17         }
18     ]
19 }
```

Loading the JSON file as Input:



Output Received:





Appendix C: Feature Weights Table

This table lists all the features used in the scoring algorithm along with their assigned weights:

Feature	Weight
COST EFFICIENCY	7
QUALITY	10
DURABILITY	8
USER EXPERIENCE	6
SUPPORT & MAINTENANCE	5
POPULARITY / TRUST	6
LONG-TERM VALUE	7
CUSTOMIZABILITY	4

This concludes the OptiPick project report.